OptiCrop – Crop Recommendation System

Project Description:

OptiCrop is a machine learning-based web application that recommends the most suitable crop based on soil nutrients and environmental conditions. The system analyzes seven important agricultural parameters: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall to predict the best crop for cultivation.

The application is developed using Python, Flask, Scikit-learn, Pandas, and NumPy, with a responsive web interface built using HTML, CSS, Bootstrap, and JavaScript. The trained machine learning model (crop_model.pkl) processes user inputs and instantly displays the recommended crop along with a brief description.

The project is deployed on Render, making it accessible through a public URL. By providing data-driven crop recommendations, OptiCrop helps farmers, agricultural students, and researchers make informed decisions, improve crop productivity, and support sustainable farming practices.

Scenarios

Scenario 1: Crop Recommendation for Paddy Farming

A farmer enters the soil nutrient values (Nitrogen, Phosphorus, Potassium), temperature, humidity, pH, and rainfall into the OptiCrop application. The system analyzes the inputs using the trained machine learning model and recommends Rice as the most suitable crop along with a brief description.

Scenario 2: Selecting the Best Crop for New Farmland

An agricultural student collects soil test results from a newly acquired field and enters the values into OptiCrop. The application predicts Maize as the most suitable crop, helping the user make an informed cultivation decision before the farming season begins.

Scenario 3: Improving Crop Productivity
A farmer wants to improve yield after experiencing poor production in the previous season. By entering updated soil and environmental parameters into OptiCrop, the system recommends Cotton as the optimal crop based on the current conditions, helping reduce the risk of crop failure and improve productivity.

Technical Architecture:

Description

OptiCrop follows a three-tier architecture consisting of a presentation layer, application layer, and machine learning layer.

- **The frontend is developed using HTML5, CSS3, Bootstrap, and JavaScript, providing a responsive interface for users to enter soil and environmental parameters.**
- **The backend is built using the Flask framework in Python, which handles user requests, validates inputs, and communicates with the trained machine learning model.**
- **The machine learning layer uses a trained Scikit-learn model (crop_model.pkl) to predict the most suitable crop based on the provided input values.**
- **The application is deployed on Render using Gunicorn, enabling users to access the system through a public web URL.**

Note: The current version of OptiCrop does not use a database. It relies on a pre-trained machine learning model and a CSV dataset for prediction. Therefore, an ER Diagram is not applicable.

Pre-requisites

- **Python Programming: Python 3.x**
- **Flask Framework: For developing the web application**
- **Machine Learning: Basic knowledge of Scikit-learn**
- **Data Processing: Pandas and NumPy**
- **Frontend Development: HTML5, CSS3, Bootstrap, JavaScript**
- **Version Control: Git and GitHub**
- **Development Environment: Visual Studio Code and Jupyter Notebook**
- **Deployment Platform: Render**
- **Package Management: pip and virtual environments**

Project Workflow:

Milestone 1: Problem Analysis, Dataset Collection & Model Selection

Activity 1.1
Collect and study the Crop Recommendation Dataset from Kaggle, containing soil nutrients and environmental parameters.
Activity 1.2
Analyze the dataset, preprocess the data, and select the most suitable Machine Learning algorithm for crop prediction.
Activity 1.3
Design the system architecture by defining the interaction between the frontend, Flask backend, and the trained machine learning model.
Activity 1.4
Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Joblib, and other required dependencies.

Milestone 2: Machine Learning Model Development

Activity 2.1
Train the machine learning model using the Crop Recommendation Dataset and evaluate its prediction accuracy.
Activity 2.2
Save the trained model (crop_model.pkl) using Joblib/Pickle for integration with the Flask application.

Milestone 3: Application Development (app.py)

Activity 3.1
Develop the main application logic in app.py by creating Flask routes, validating user inputs, loading the trained machine learning model, processing prediction requests, and displaying the recommended crop with its description.

Milestone 4: Frontend Development

Activity 4.1
Design and develop a responsive web interface using HTML, CSS, Bootstrap, and JavaScript for entering soil and environmental parameters.
Activity 4.2
Integrate Flask templates (render_template) to dynamically display the predicted crop and its description based on user input.

Milestone 5: Deployment

Activity 5.1
Prepare the application for deployment by configuring the project structure, installing all required dependencies from requirements.txt, and testing the application locally.
Activity 5.2
Deploy the Flask application on Render, configure the build and start commands, and verify that

the deployed application is publicly accessible.

Milestone 6: Testing, Documentation & Conclusion

Activity 6.1

Perform functional testing using different combinations of soil and environmental parameters to verify prediction accuracy.

Activity 6.2

Prepare the project documentation, deployment details, screenshots, GitHub repository, and demonstration of the deployed application.

Conclusion

OptiCrop successfully predicts the most suitable crop based on soil nutrients and environmental conditions using a machine learning model. The project demonstrates the practical application of machine learning in agriculture through a user-friendly Flask web application deployed on Render. Future enhancements may include weather API integration, fertilizer recommendations, irrigation suggestions, and multilingual support to further assist farmers in making informed agricultural decisions.

Milestone 1: Model Selection and System Architecture

In this milestone, we focus on selecting the appropriate machine learning model for crop prediction and designing the overall architecture of the OptiCrop application. This involves collecting and analyzing the crop recommendation dataset, selecting a suitable machine learning algorithm, preparing the development environment, and defining the interaction between the frontend, backend, and trained model. These activities establish the foundation for building a reliable crop recommendation system.

Activity 1.1: Collect the Crop Recommendation Dataset

Before developing the machine learning model, a suitable dataset containing soil and environmental parameters is required.

Step 1 — Download the Dataset

Visit Kaggle and download the Crop Recommendation Dataset.

Step 2 — Study the Dataset

Analyze the dataset to understand the available input features and output labels.

Input Features

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

Output

- **Recommended Crop**

Step 3 — Data Preprocessing

Prepare the dataset for machine learning by:

- **Checking for missing values**
- **Removing duplicate records**
- **Verifying data types**
- **Splitting the dataset into training and testing datasets**

Step 4 — Save the Dataset
Store the dataset as
Crop_recommendation.csv
inside the project directory for model training.

Activity 1.2: Research and Select the Appropriate Machine Learning Model
To select the best machine learning model, the following activities were performed.
Understand the Project Requirements
The objective of OptiCrop is to recommend the most suitable crop using soil nutrients and environmental conditions.

Analyze the Dataset
Study the relationship between soil parameters and crop labels using exploratory data analysis.

Evaluate Different Machine Learning Algorithms
Compare different machine learning algorithms based on prediction accuracy and performance.
Examples include:

- **Decision Tree**
- **Random Forest**
- **Support Vector Machine**
- **K-Nearest Neighbors**

Select the Best Model
Choose the model that provides the highest prediction accuracy.
Train the selected model using Scikit-learn and save it as crop_model.pkl
for use in the Flask application.

Activity 1.3: Define the Architecture of the Application
Create the System Architecture
Design the architecture of the OptiCrop application consisting of:

- **Frontend**
- **Flask Backend**
- **Machine Learning Model**

Frontend Functionality
The frontend allows users to:

- **Enter Nitrogen value**
- **Enter Phosphorus value**
- **Enter Potassium value**
- **Enter Temperature**
- **Enter Humidity**
- **Enter pH**
- **Enter Rainfall**
- **Click Predict Crop**

Backend Responsibilities
The Flask backend performs the following tasks:

- **Receives user input**
- **Validates the entered values**
- **Converts input into a Pandas DataFrame**
- **Loads the trained machine learning model**
- **Sends input to the model**
- **Receives the predicted crop**
- **Displays the crop recommendation and description**

Machine Learning Integration
The trained model (crop_model.pkl) is loaded during application startup.
Whenever the user submits the form:

- **Input data is processed.**
- **The machine learning model predicts the most suitable crop.**
- **The prediction is displayed on the webpage.**

Activity 1.4: Set Up the Development Environment
Install Python
Install Python 3.x on the development system.

Create a Virtual Environment
Create and activate a virtual environment to isolate project dependencies.

Install Required Libraries

Install all required packages using:

pip install -r requirements.txt

The project uses the following libraries:

- **Flask**
- **Pandas**
- **NumPy**
- **Scikit-learn**
- **Joblib**
- **Gunicorn**

Configure the Project Structure

Organize the project into the following structure:

```
OptiCrop/
│
├── app.py
├── crop_model.pkl
├── requirements.txt
├── README.md
│
├── dataset/
│   └── notebook/
│       ├── Crop_recommendation.csv
│       ├── templates/
│       ├── static/
│       └── flask_app.py
```

Verify the Application

Run the Flask application locally and verify that:

- **The home page loads successfully.**
- **Users can enter soil parameters.**
- **The machine learning model predicts the appropriate crop.**
- **The recommended crop and description are displayed correctly.**

Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the OptiCrop – Crop Recommendation System are developed. The application accepts soil nutrient and environmental parameters from the user, processes the data using the trained machine learning model, and displays the most suitable crop along with its description through a user-friendly web interface.

Activity 2.1: Develop Core Crop Prediction Features

Implement the core functionalities required for crop recommendation.

Soil Parameter Input

Develop an input form that allows users to enter the following agricultural parameters:

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

Crop Prediction

After receiving the user inputs:

- **Validate the entered values.**
- **Convert the input into a Pandas DataFrame.**
- **Send the processed data to the trained machine learning model (crop_model.pkl).**
- **Predict the most suitable crop based on the given parameters.**

Crop Information Display

After prediction:

- **Display the recommended crop.**
- **Show a brief description of the predicted crop.**
- **Present the result in a clear and user-friendly format.**

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

Configure the main route (/) in app.py to display the homepage and process crop prediction requests.

Process User Input

Create an input form in index.html where users can enter the required soil and environmental parameters.

The Flask backend uses request.form to retrieve the submitted values:

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

Validate that:

- **All input fields are completed.**
- **All values are numeric.**
- **Invalid or empty inputs generate an appropriate error message.**

Integrate the Machine Learning Model

Within the prediction route:

- **Load the trained machine learning model (crop_model.pkl).**
- **Convert user input into a Pandas DataFrame.**
- **Pass the data to the model using the predict() method.**
- **Receive the predicted crop from the model.**
- **Retrieve the corresponding crop description.**

- **Display the prediction and description on the webpage.**

Display Prediction Results

Once the prediction is completed:

- **Return the predicted crop.**
- **Display the crop description.**
- **Render the output dynamically using Flask's render_template() function.**

Deliverables

- **Functional crop prediction module.**
- **Flask backend for handling user requests.**
- **Machine learning model successfully integrated.**
- **Dynamic display of crop recommendations.**
- **User-friendly web interface for entering soil parameters and viewing results.**

Top of Form

Bottom of Form

Milestone 3: App.py Development

Milestone 3 focuses on developing the core logic of the app.py file, which serves as the backend of the OptiCrop application. In this milestone, the Flask application is configured to receive user inputs, validate the entered soil and environmental parameters, load the trained machine learning model, perform crop prediction, and display the recommendation through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Route

Establish the main route for the application:

/

- **Displays the home page (index.html).**
- **Accepts both GET and POST requests.**

- **Processes crop prediction requests submitted through the web form.**

Set Up the Route Handler

The route handler performs the following operations:

- **Receives user inputs using request.form.**
- **Reads the following parameters:**
  - **Nitrogen (N)**
  - **Phosphorus (P)**
  - **Potassium (K)**
  - **Temperature**
  - **Humidity**
  - **pH**
  - **Rainfall**
- **Validates that all input values are numeric and not empty.**
- **Displays an error message if invalid input is detected.**

Load the Machine Learning Model

- **Load the trained machine learning model (crop_model.pkl) when the application starts.**
- **Use Joblib as the primary loader, with Pickle as a fallback if required.**
- **Keep the model loaded in memory to improve prediction performance.**

Process Prediction Requests

When the user submits the form:

- **Convert the input values into a Pandas DataFrame.**
- **Pass the processed data to the trained Scikit-learn model using the predict() method.**
- **Receive the predicted crop from the model.**
- **Match the predicted crop with its description stored in the application.**

Display Prediction Results

After prediction:

- **Display the recommended crop.**
- **Display a brief description of the predicted crop.**
- **Render the results dynamically using Flask's render_template() function.**

If any error occurs during processing:

- **Display an appropriate error message to the user.**

Flask Route Summary

Home Route

/

- **Renders the index.html page.**
- **Displays the crop prediction form.**
- **Accepts user input.**
- **Processes crop prediction requests.**
- **Returns the recommended crop and crop description.**

Input Parameters

The application accepts the following user inputs:

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

Output

The application returns:

- **Recommended Crop**
- **Crop Description**

Deliverables

- **Flask application (app.py) developed successfully.**
- **Machine learning model integrated with the Flask backend.**
- **Input validation implemented.**
- **Crop prediction functionality completed.**
- **Dynamic result display using Flask templates.**
- **Error handling implemented for invalid inputs.**

Top of Form

Bottom of Form

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly interface for the OptiCrop – Crop Recommendation System. The objective is to create an intuitive web application where users can easily enter soil and environmental parameters and receive instant crop recommendations. The frontend is developed using HTML, CSS, Bootstrap, and JavaScript, while Flask templates are used to dynamically display the prediction results.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- **Develop the main HTML file (index.html) as the primary interface of the application.**
- **Include a project title, navigation header, input form, prediction section, and footer.**
- **Use semantic HTML elements to improve readability, accessibility, and maintainability.**
- **Organize the interface into clearly defined sections for user input and prediction results.**

Design a Responsive Layout Using CSS

- **Create a responsive user interface using CSS3 and Bootstrap.**

- **Design a clean layout with proper spacing, typography, and color combinations.**
- **Use Bootstrap's grid system to ensure compatibility across desktops, tablets, and mobile devices.**
- **Add buttons, input fields, and cards to improve the overall user experience.**

Create User Interface Components

Develop separate UI components for the application:

Input Form

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

Prediction Section

Display:

- **Recommended Crop**
- **Crop Description**

Action Button

Provide a Predict Crop button that submits the entered values to the Flask backend for processing.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- **Create an HTML form that collects all seven agricultural parameters.**
- **Submit the form to the Flask backend using the POST method.**
- **Validate the entered values before processing the prediction.**

Display Prediction Results

After receiving the prediction from the Flask backend:

- **Display the recommended crop.**
- **Display a brief description of the recommended crop.**
- **Preserve the entered values in the form after prediction.**
- **Present the results in a clear and visually appealing card or result section.**

Improve User Experience

- **Display meaningful error messages when invalid or incomplete values are entered.**
- **Keep the interface responsive and easy to navigate.**
- **Ensure fast loading and smooth interaction between the frontend and backend.**

Deliverables

- **Responsive homepage (index.html) developed successfully.**
- **User-friendly input form for soil and environmental parameters.**
- **Bootstrap-based responsive layout.**
- **Dynamic display of crop prediction results.**
- **Integration of Flask templates for rendering predictions.**
- **Improved user experience with input validation and clear result presentation.**

Milestone 5: Deployment

Milestone 5 focuses on deploying the OptiCrop – Crop Recommendation System. The application is first tested locally to verify its functionality and then deployed publicly on Render to make it accessible through a web browser. This milestone includes configuring the project environment, installing dependencies, testing the application, and

publishing it online.

Activity 5.1: Preparing the Application for Deployment

Set Up a Virtual Environment

- **Open Visual Studio Code or Command Prompt.**
- **Navigate to the project directory.**
- **Create and activate a Python virtual environment.**

Install Required Dependencies

Install all required libraries using the following command:

pip install -r requirements.txt

The required packages include:

- **Flask**
- **Pandas**
- **NumPy**
- **Scikit-learn**
- **Joblib**
- **Gunicorn**

Verify Project Structure

Ensure the following files are present in the project directory:

- **app.py**
- **requirements.txt**
- **crop_model.pkl**
- **README.md**
- **templates/index.html**
- **static/**
- **dataset/notebook/Crop_recommendation.csv**

Configure the Flask Application

Verify that:

- **The Flask application object is named app.**
- **The machine learning model (crop_model.pkl) loads**

successfully.

- All required dependencies are installed.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the application locally using:

python app.py

Open the Application

Open a web browser and navigate to:

http://127.0.0.1:5000

Test the Application

Verify the following:

- **The home page loads successfully.**
- **The input form accepts all seven parameters:**
  - **Nitrogen (N)**
  - **Phosphorus (P)**
  - **Potassium (K)**
  - **Temperature**
  - **Humidity**
  - **pH**
  - **Rainfall**
- **Clicking the Predict Crop button displays the recommended crop.**
- **The crop description is displayed correctly.**
- **Invalid inputs generate appropriate error messages.**

Activity 5.3: Public Deployment on Render

Render is a cloud hosting platform used to deploy the Flask application and make it publicly accessible.

Push the Project to GitHub

Upload the complete project to a GitHub repository.

Create a Render Web Service

- **Log in to Render.**
- **Connect the GitHub repository.**
- **Create a new Web Service.**
- **Select the OptiCrop repository.**

Configure Build Settings

Build Command

pip install -r requirements.txt

Start Command

gunicorn app:app

Deploy the Application

Click Deploy and wait for Render to build and deploy the application successfully.

Access the Deployed Application

After deployment, the application is available at:

https://opticrop-wii6.onrender.com/

Users can access the application directly through this URL without installing any software.

Important Notes

- **Ensure requirements.txt contains all required Python packages before deployment.**
- **Keep the trained model file (crop_model.pkl) in the correct project location.**
- **Verify that the Flask application object is named app so that Gunicorn can start the application successfully.**
- **Any changes made to the source code should be committed and pushed to GitHub before redeploying the application on Render.**
- **The deployed application can be updated automatically**

whenever new changes are pushed to the connected GitHub repository.

Deliverables

- **Virtual environment configured successfully.**
- **Application tested locally.**
- **Project uploaded to GitHub.**
- **Flask application deployed on Render.**
- **Public deployment URL generated and verified.**
- **Users can access the OptiCrop application online for crop prediction.**

Top of Form

Bottom of Form

Exploring the Website's Web Pages

Home Page

Description:

The Home Page serves as the main interface of the OptiCrop – Crop Recommendation System. It provides a simple and responsive design where users can enter soil nutrient and environmental parameters to receive crop recommendations. The page contains a project header, a crop prediction form, and a prediction result section. The clean layout allows users to interact with the application easily from desktop and mobile devices.

Input Section

Description:

The Input Section contains a user-friendly form where users enter the agricultural parameters required for crop prediction.

The input fields include:

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**

After entering the required values, users click the Predict Crop button. The entered data is sent to the Flask backend, where it is validated and processed by the trained machine learning model.

Prediction Results Section

Description:

After successful prediction, the application displays the results dynamically on the same page.

The Results Section displays:

- **Recommended Crop**
- **Crop Description**

If invalid or incomplete input values are entered, the application displays an appropriate error message, helping users provide correct information for accurate predictions.

Conclusion

OptiCrop – Crop Recommendation System is a machine learning-based web application developed using Python, Flask, and Scikit-learn to recommend the most suitable crop based on soil nutrients and environmental conditions. The system analyzes seven important agricultural parameters—Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall—to provide accurate crop recommendations along with a brief description of the predicted crop.

The project includes dataset collection, machine learning model training, Flask-based backend development, responsive frontend design using HTML, CSS, and Bootstrap, and successful deployment on Render. The application demonstrates how machine learning can support data-driven agricultural decision-making and help farmers improve crop selection.

In the future, OptiCrop can be enhanced by integrating real-time weather data, fertilizer recommendations, soil health analysis, IoT sensor integration, multilingual support, and user authentication to provide a more comprehensive smart agriculture solution. These improvements will further increase the usefulness of the application for farmers, agricultural researchers, and students.

Top of Form

Bottom of Form